HOCHSCHULE ANHALT University of Applied Sciences

Distributed Software Architecture

Lab 1 · Semester project

Your First Service: Requests, Containers and State

Prof. Dr. Dmitry Kachan



EMW, Hochschule Anhalt

Summer Semester 2026

A service you can explain

Your semester project begins with one small service. Someone sends a request, the service changes an item, and a reply comes back. You will make this work, then investigate what happens when its address or running process changes.

Guiding question

Can you explain what happens when someone uses your service—and when it restarts?



Three ideas to carry with you

- A request, response and resulting state must agree.
- An address reaches a listener through a particular port mapping.
- Application memory belongs to the running process.

Choose your story; keep one API

Story	Example name / category	Allowed status
Campus tasks	Prepare presentation / study	`open`, `doing`, `done`
Greenhouse catalogue	Basil / herb	`healthy`, `warning`, `retired`
Board-game library	Catan / strategy	`available`, `on_loan`, `maintenance`

All stories use `id`, `name`, `status` and `category`, and the path `/items`. The service generates the ID. Choose your own records. A library status is descriptive; this lab does not implement loan coordination.

Four checkpoints, one growing repository

1. **Meet your service:** run it and trace a request.
2. **Create something valid:** compare accepted and rejected input.
3. **Follow one item:** replace it, delete it, then look again.
4. **Move and restart it:** investigate Docker, ports and memory.

What is provided? Working health/read routes, JSON/error helpers, an annotated Dockerfile, dependency files and staged checks. You implement validation, creation, replacement and deletion. Initially the unfinished routes deliberately return **501**.

AI assistance is welcome. First make a brief personal prediction; then use code, documentation and tools to test it. Record four short entries in `docs/observations.md`; keep your story in `docs/story.md`. No additional long report is required.

Setup, selecting a story and the two-terminal workflow

Open this guide inside the student ZIP to use its relative help links. Commands below use macOS/Linux syntax; PowerShell equivalents are in the setup help.

1 · Meet your service

Which process receives the request, and where would its data live?

If setup already started your service, keep it running. Otherwise activate the virtual environment and, in terminal A, start from the project root:

</> Start the local service

```
cd catalog-service
waitress-serve --host=127.0.0.1 --port=5000 --threads=1 app:app
```

`app:app` means the application object `app` in `app.py`. Waitress runs the application. This lab uses one process and one application-processing thread; the dictionary is not a general concurrent database.

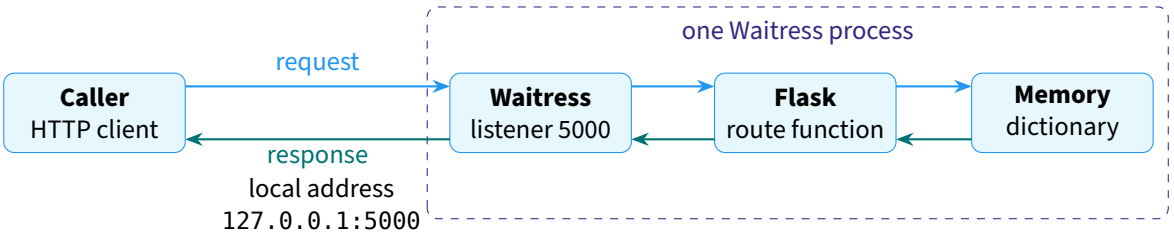
Predict Before calling it: what would an empty collection return? What should a lookup of item 99 return? Would a working health route prove that creation is implemented?

Observe In terminal B, from the project root with the same environment activated:

</> Inspect the supplied routes

```
curl -i http://127.0.0.1:5000/health
curl -i http://127.0.0.1:5000/items
curl -i http://127.0.0.1:5000/items/99
python checks/check_service.py --stage starter
```

Compare status, response body and state. The staged checker confirms the starting point, including deliberate 501 responses; it does not certify completed CRUD.



For this local run, the caller reaches Waitress directly at `127.0.0.1:5000`; Docker is added at checkpoint 4. Flask selects a route. The dictionary is inside this same process.

Change Choose `PROJECT_STORY` using the setup instructions. Restart the local service, check the story in `/health`, and pick one example item. Study the supplied read route and `parse_object` helper in `app.py`.

Explain Point to the listener, the route and the state owner. Explain what 200, 404 and the starter's 501 each establish. Save your first observation entry.

No response, occupied port or missing dependencies?

2 · Create something valid

Does rejecting a request leave the state unchanged?

Predict An item has a name, a story-specific status and a category. Predict the status and resulting collection for a valid item and for an item whose name is empty.

Change Complete `validate_item` and `POST /items` in `app.py`. Use the supplied JSON helper before checking the business fields. Restart Waitress after changing Python source; it does not reload your file automatically.

The writable item contract

Accept exactly `name`, `status` and `category`. All must be strings. Trim surrounding whitespace. Name and category must be nonempty; status must belong to your story. Reject missing fields, extra fields and a client-supplied `id`. Generate a positive ID in the service. A rejected mutation must not change the dictionary or consume an ID.

</> A complete example request (greenhouse story)

```
curl -i -X POST http://127.0.0.1:5000/items \
  -H 'Content-Type: application/json' \
  -d '{"name":"Basil","status":"healthy","category":"herb"}'
```

Use a valid status for your chosen story. Creation returns **201** and the created object, including its generated ID. Read that ID back through `GET /items/{id}`.

Observe Repeat the experiment with an empty name. Also try malformed JSON and the wrong media type. Read the collection after rejection: do not judge state from the error message alone.

Request	Status	State effect
Valid creation	201	One new item with generated ID
Invalid fields / malformed or non-object JSON	400	No change
Unsupported media type	415	No change

</> Check this checkpoint

```
python checks/check_service.py --stage create
```

Explain What is the difference between parsing JSON and validating an item? Which component generates identity? Use an observed rejection to explain how you know that no item was created. Save your second entry.

Exact error shape, status codes and worked lookup pattern

3 · Follow one item

Do the response and the resulting state agree?

At the start of session 2, sketch the request path from memory. Compare it with your running service and correct the sketch before continuing.

Predict You replace an existing item, then delete it. Which fields may change? What must happen to the ID? What should the next lookup return?

Change Complete PUT `/items/{id}` and DELETE `/items/{id}`. PUT replaces *all three writable fields* of an existing item and preserves its ID. It does not create a missing item. Reuse the same validation as creation.

Operation	Status	Result
Read / replace an existing item	200	Return the item; replacement retains ID
Delete an existing item	204	Empty response body
Read, replace or delete a missing item	404	No state change
Rejected replacement	400/415	Existing item remains unchanged

Observe Create one item and write down its ID. Replace its name and category with your own valid example; read it back. Try a replacement with a missing field and compare the item before and after. Delete it, look it up, then repeat deletion.

</> One replacement example: substitute your created ID

```
curl -i -X PUT http://127.0.0.1:5000/items/1 \
  -H 'Content-Type: application/json' \
  -d '{"name":"Mint","status":"healthy","category":"herb"}'
curl -i http://127.0.0.1:5000/items/1
curl -i -X DELETE http://127.0.0.1:5000/items/1
curl -i http://127.0.0.1:5000/items/1
python checks/check_service.py --stage crud
```

The checks create their own records. Their sample ID is not necessarily yours. They verify both replacement and deletion, including state after rejected mutations.

Explain Why does an unchanged ID matter? Why is 204 followed by a successful lookup inconsistent with this contract? How does your code ensure that a rejected replacement leaves the old item intact? Save your third entry.

Ready for the next checkpoint: all CRUD checks pass locally. Stop the local Waitress process with Ctrl+C before investigating the container's addresses.

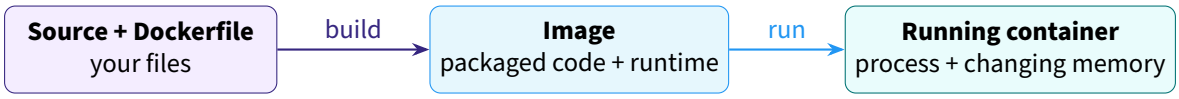
Mutation rules and error-response examples

4 · Move and restart it

Which changes affect reachability, and which affect state?



Docker packages and runs your service. Read the supplied Dockerfile: identify the base image, copied files, dependencies, non-root user and startup command. The container listens on `0.0.0.0:5000`; EXPOSE documents a port but does not publish it.



Change From the project root, build and run. Confirm that the name and ports are free; if occupied, use the troubleshooting help. After `docker run`, wait until the health route answers 200 before running the checker.

</> Build and run your container

```
docker build -t dsa26-lab1:1.0 catalog-service
docker run -d --name dsa26-lab1-student \
  -p 127.0.0.1:8085:5000 dsa26-lab1:1.0
curl -i http://127.0.0.1:8085/health
python checks/check_service.py --stage crud \
  --base-url http://127.0.0.1:8085
docker logs dsa26-lab1-student
```

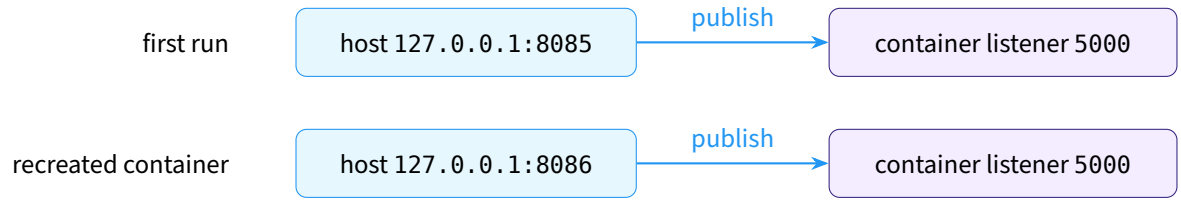
For campus or library, add `-e PROJECT_STORY=campus` or `library` before the image name. Use the same selection when recreating it.

Predict Create an item and read it back. What will happen to it after stop/start of this *same container*? Record your prediction before running:

</> Restart the process and inspect its state

```
docker stop dsa26-lab1-student
docker start dsa26-lab1-student
curl -i http://127.0.0.1:8085/items
```

Observe Wait for the health route to answer; read the collection again. Does the result support your prediction? This lab uses application memory, not file-backed storage.



Change Now recreate *your* container using `127.0.0.1:8086:5000`; the complete commands are in the Docker help. Predict the caller's new address and verify it. Container port 5000 remains the same. Publication changes require recreation; changing a port is not the cause of memory loss.

Explain Trace the host-to-container request, distinguish image/container/process, and explain both observations. Save your fourth entry; remove only your container when finished.

Full recreation commands, identity check and owned-resource cleanup

Bring your service and your understanding

Your repository is the foundation for the next milestone. Keep its source, Dockerfile, chosen story and short observations. The items currently exist only in application memory.



Before opening the help again, explain:

1. Why can the caller use port 8086 while Waitress still listens on 5000?
2. How do you establish that invalid input did not change the data?
3. Why does restarting lose items, even when the image is unchanged?

One acceptance checklist

- ☐ Health/read routes work; creation generates an ID and returns 201.
- ☐ Replacement preserves the ID; both update and delete are verified.
- ☐ Deletion returns 204 with no body; later lookup/deletion returns 404.
- ☐ Invalid fields, JSON and media type are rejected; state is unchanged.
- ☐ The container serves the same contract through the published port.
- ☐ You observed restart behavior and verified the changed host port.
- ☐ `docs/story.md` and four short observation entries are ready.

Your existing interview

Run your service and demonstrate one normal path and a meaningful rejected request. Explain where the state lives and what each response establishes. Be ready to predict a familiar changed condition, inspect its result and explain what happened. Use your working repository; a separate screenshot dossier or long report is not required.

Two sessions plus work at home

When	Focus
Session 1 · 90 min	Choose story 10 min; meet service 20; creation 45; feedback 15.
Between sessions	Finish creation; implement and check PUT/DELETE.
Session 2 · 90 min	Recall 10 min; follow item 25; Docker 40; explain/prepare 15.
Afterward	Resolve remaining failures; complete observations. Record actual effort; these allocations are planning budgets.

Read when you need it

Flask Quickstart: routing, request data and responses.

Docker: publishing ports: host/container ports and EXPOSE.

Waitress options: listener address and application threads.

Support index: setup, contract, Docker and troubleshooting